

Lab 4: Pipeline Orchestration with Apache Airflow

- **Course:** Engineering of Intelligent Models
- **Module:** M3. Model Orchestration & Automation
- **Focus Tool:** Apache Airflow
- **Format:** Hands-on Laboratory
- **Branch:** Lab4

1. Goal of the Lab

The objective of this lab is to orchestrate the ingestion, training, and evaluation phases of our **Weather Forecast Capstone Project** into a logical sequence.

You will learn how to:

1. Define workflows as Directed Acyclic Graphs (DAGs) to ensure idempotency and fault tolerance.
2. Expand your Docker Compose environment to include Airflow's core components (Webserver, Scheduler, and Metadata Database).
3. Write a Python definition file that connects your Hydra-configured Python scripts into a linear pipeline.
4. **(Preparation for Lab 5):** Create the necessary task stubs for the upcoming model evaluation phase.

2. Apache Airflow: Why Orchestration?

In a production machine learning environment, data arrives continuously, and models degrade over time (*data drift*). Relying on a human engineer to run `python get_data.py` followed by `python train.py` is unscalable and prone to error.

Apache Airflow solves this by acting as the "conductor" of your MLOps orchestra.

- **Directed Acyclic Graph (DAG):** Workflows are represented as graphs where nodes are tasks and edges are dependencies. "Acyclic" guarantees the pipeline flows strictly forward. It cannot enter an infinite loop.
- **Consistency:** Airflow encourages designing tasks that, if executed multiple times with the same inputs, yield the exact same end state. Our integration of DVC (Lab 2) and Hydra (Lab 3) ensures our scripts are natively idempotent.

3. Version Control: Switching Branches

Before altering our infrastructure, ensure your repository is clean and branched.

Switch to a new branch for Lab 4.

```
In [ ]: !git add .
!git commit -m "Complete Lab 3: Hydra dynamic configuration"

# Create and switch to the Lab4 branch
!git checkout -b Lab4
```

4. Infrastructure Evolution: Extending Docker

To run Airflow alongside MLflow, we must expand our infrastructure. Because Airflow needs our specific Python dependencies to execute our scripts, we cannot use the vanilla Airflow image. We must build a custom one.

Step 1: Add `apache-airflow` to requirements

First, ensure you have `apache-airflow` in your `requirements.txt` so that it gets installed in the Airflow container.

```
# requirements.txt
apache-airflow==3.1.7
```

Step 2: Create a `Dockerfile`

In the `dags/` folder, create a file named `Dockerfile` with the following content:

```
FROM apache/airflow:3.1.7
USER root
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        vim \
    && apt-get autoremove -yqq --purge \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

USER airflow
# Copy requirements and install them in the Airflow container
COPY requirements.txt /requirements.txt
RUN pip install --no-cache-dir -r /requirements.txt
# Set the safe directory for git to avoid issues with Airflow's DVC integration
RUN git config --global --add safe.directory '*'
```

Code Explanation:

- `FROM apache/airflow:3.1.7` : We start from the official Airflow image, which includes all the necessary components to run Airflow.
- `USER root` and `USER airflow` : We switch to the root user to install additional packages (in this case, only `vim` for now) and then switch back to the `airflow` user, which is the default user in the Airflow image.
- `COPY requirements.txt /requirements.txt` : We copy our project's `requirements.txt` into the container so that we can install our specific Python dependencies.
- `RUN pip install --no-cache-dir -r /requirements.txt` : This command installs all the Python dependencies listed in `requirements.txt` inside the Airflow container.
- `RUN git config --global --add safe.directory '*'` : This command configures Git to treat all directories as safe, which is necessary for Airflow to interact with our DVC-tracked repository without running into permission issues.

Step 3: Update `docker-compose.yaml`

Before we add the Airflow services, we need to ensure that the MLflow server allows connections from the Airflow containers. To do this, add the following environment variable to your existing `mlflow_server` service in `docker-compose.yaml` :

```
- MLFLOW_SERVER_ALLOWED_HOSTS=mlflow_server:*,localhost:*,127.0.0.1:*
```

Now, append the Airflow services to your existing `docker-compose.yaml` . This establishes the **PostgreSQL** backend, the **Scheduler**, and the **Web UI** and the **DAG Processor**, while mapping your local `dags/` , `src/` , `conf/` , and `data/` folders directly into the containers.

```
# Add these services below your existing mlflow_server service
```

```
postgres:
  container_name: emi-postgres
  image: postgres:18.3-alpine3.23
  restart: unless-stopped
  environment:
    - POSTGRES_USER=airflow
    - POSTGRES_PASSWORD=airflow
    - POSTGRES_DB=airflow
  volumes:
    - emi-postgres_data:/var/lib/postgresql/18.3/docker

airflow-init:
  container_name: emi-airflow-init
  build:
    context: .
    dockerfile: dags/Dockerfile
  depends_on:
    - postgres
  environment:
    - AIRFLOW__CORE__EXECUTOR=LocalExecutor
  -
```

```

AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/airflow
- AIRFLOW__API__AUTH__JWT_SECRET=super_secret_mlops_jwt_key
- AIRFLOW__API__SECRET_KEY=super_secret_mlops_api_key
command: airflow db migrate

airflow-webserver:
  container_name: emi-airflow-webserver
  build:
    context: .
    dockerfile: dags/Dockerfile
  depends_on:
    - postgres
  restart: unless-stopped
  ports:
    - "8080:8080"
  environment:
    - AIRFLOW__CORE__EXECUTOR=LocalExecutor
    -
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/airflow
- AIRFLOW__CORE__LOAD_EXAMPLES=False
- AIRFLOW__CORE__SIMPLE_AUTH_MANAGER_ALL_ADMINS=True # Make all users admins for local
development
- AIRFLOW__CORE__EXECUTION_API_SERVER_URL=http://airflow-webserver:8080/execution/
- AIRFLOW__API__AUTH__JWT_SECRET=super_secret_mlops_jwt_key
- AIRFLOW__API__SECRET_KEY=super_secret_mlops_api_key
volumes:
- ./dags:/opt/airflow/dags
- ./src:/opt/airflow/src
- ./conf:/opt/airflow/conf
- ./data:/opt/airflow/data
- ./dvc:/opt/airflow/.dvc
- ./git:/opt/airflow/.git
command: airflow api-server

airflow-scheduler:
  container_name: emi-airflow-scheduler
  build:
    context: .
    dockerfile: dags/Dockerfile
  depends_on:
    - postgres
  restart: unless-stopped
  environment:
    - AIRFLOW__CORE__EXECUTOR=LocalExecutor
    -
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/airflow
- AIRFLOW__CORE__EXECUTION_API_SERVER_URL=http://airflow-webserver:8080/execution/
- AIRFLOW__API__AUTH__JWT_SECRET=super_secret_mlops_jwt_key
- AIRFLOW__API__SECRET_KEY=super_secret_mlops_api_key
volumes:
- ./dags:/opt/airflow/dags
- ./src:/opt/airflow/src
- ./conf:/opt/airflow/conf
- ./data:/opt/airflow/data
- ./dvc:/opt/airflow/.dvc
- ./git:/opt/airflow/.git
command: airflow scheduler

airflow-dag-processor:
  container_name: emi-airflow-dag-processor
  build:
    context: .
    dockerfile: dags/Dockerfile
  depends_on:
    postgres:
      condition: service_started
    airflow-init:
      condition: service_completed_successfully
  restart: unless-stopped
  environment:

```

```

- AIRFLOW__CORE__EXECUTOR=LocalExecutor
-
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/airflow
- AIRFLOW__API__AUTH__JWT_SECRET=super_secret_mlops_jwt_key
- AIRFLOW__API__SECRET_KEY=super_secret_mlops_api_key
volumes:
- ./dags:/opt/airflow/dags
- ./src:/opt/airflow/src
- ./conf:/opt/airflow/conf
- ./data:/opt/airflow/data
- ./dvc:/opt/airflow/.dvc
- ./git:/opt/airflow/.git
command: airflow dag-processor

volumes:
  emi-postgres_data:

```

Code Explanation:

- postgres** : This service sets up a PostgreSQL database that Airflow will use to store its metadata, including DAG definitions, task states, and logs.
 - environment** : We set the database credentials and name for Airflow to connect to. You can customize these values, but ensure they match the connection string used in the Airflow services.
 - emi-postgres_data:/var/lib/postgresql/18.3/docker** : We create a named volume to persist the PostgreSQL data, ensuring that our Airflow metadata is not lost when the container restarts. This maps the named volume `emi-postgres_data` to the PostgreSQL data directory inside the container.
- airflow-init** : This service is responsible for initializing the Airflow database schema.
 - build** : We specify the build context and Dockerfile, which is located in the `dags/` directory. This Dockerfile installs our project dependencies into the Airflow image.
 - depends_on** : It depends on the `postgres` service to ensure the database is ready before initialization. This service will run once and then exit, as its sole purpose is to prepare the database for Airflow.
 - environment** : We set the necessary environment variables for Airflow to connect to the PostgreSQL database and configure authentication for the Airflow API.
 - AIRFLOW__CORE__EXECUTOR=LocalExecutor** tells Airflow to use the LocalExecutor, which is suitable for development and testing environments. It allows tasks to run in parallel on the same machine.
 - AIRFLOW__DATABASE__SQL_ALCHEMY_CONN** specifies the connection string for Airflow to connect to the PostgreSQL database.
 - AIRFLOW__API__AUTH__JWT_SECRET** and **AIRFLOW__API__SECRET_KEY** are set to secure the Airflow API with JWT authentication.
 - command: airflow db migrate** : Runs the command to initialize the Airflow database schema. This will create the necessary tables and structures in the PostgreSQL database for Airflow to function properly.
- airflow-webserver** : This service runs the Airflow web server, which provides the user interface for monitoring and managing DAGs. It depends on the `postgres` service to ensure the database is available before starting.
 - environment** : Similar to the `airflow-init` service, we set environment variables for database connection and API authentication. And additionally:
 - AIRFLOW__CORE__LOAD_EXAMPLES=False** disables the loading of example DAGs, keeping our Airflow instance clean and focused on our project.
 - AIRFLOW__CORE__SIMPLE_AUTH_MANAGER_ALL_ADMINS=True** makes all users admins for local development, simplifying access to the Airflow UI without needing to set up individual user accounts.
 - AIRFLOW__CORE__EXECUTION_API_SERVER_URL=http://airflow-webserver:8080/execution/** configures the URL for the Airflow API server, which is necessary for task execution and monitoring. The naming `airflow-webserver` corresponds to the service name defined in this `docker-compose.yml`, allowing other services to communicate with it using this hostname.
 - ports** : We map port 8080 of the container to port 8080 on the host machine, allowing us to access the Airflow UI via `http://localhost:8080`.
 - volumes** : We mount our local project directories (`dags/`, `src/`, `conf/`, `data/`, `.dvc/`, and `.git/`) into the container. This allows Airflow to access our DAG definitions, Python scripts, configuration files, data, and version control metadata directly. You need to explicitly mount the `.dvc` and `.git` directories to

ensure that Airflow can interact with DVC for data versioning and Git for any version control operations within the tasks.

- `command: airflow api-server` : This command starts the Airflow API server, which is responsible for handling API requests related to task execution and monitoring.
- `airflow-scheduler` : This service runs the Airflow scheduler, which is responsible for scheduling and executing tasks based on the defined DAGs. It also depends on the `postgres` service to ensure the database is available before starting.
 - `environment` : Similar to the other Airflow services, we set environment variables for database connection and API authentication. The scheduler also needs to know the URL of the Airflow API server to communicate with it for task execution.
 - `volumes` : We mount the same project directories as in the webserver to ensure that the scheduler has access to all necessary files for task execution.
 - `command: airflow scheduler` : This command starts the Airflow scheduler process, which continuously monitors the DAGs and triggers tasks according to their schedules and dependencies.
- `airflow-dag-processor` : This service runs the Airflow DAG processor, which is responsible for parsing and processing the DAG files.
 - `depends_on` : It depends on both the `postgres` service (to ensure the database is available) and the `airflow-init` service (to ensure the database schema is initialized) before starting. This ensures that the DAG processor can properly register DAGs in the Airflow metadata database.
 - `environment` : Similar to the other Airflow services, we set environment variables for database connection and API authentication.
 - `volumes` : We mount the same project directories to ensure that the DAG processor has access to all necessary files for parsing and processing DAGs.
 - `command: airflow dag-processor` : This command starts the Airflow DAG processor, which continuously monitors the `dags/` directory for changes and processes any new or updated DAG files.

Finally, we define a named volume `emi-postgres_data` to persist PostgreSQL data across container restarts, ensuring that our Airflow metadata is not lost.

Step 3: Launch the updated infrastructure

```
In [ ]: # Change to the root directory of your project where the docker-compose.yaml is located if you're running
import os
os.chdir("../..")

# Force a rebuild of the image to install requirements, then launch
!docker compose up --build -d
```

You can now access the Airflow UI at `http://localhost:8080`. The first time you access it, you may see a message about the database being initialized. This is normal as the `airflow-init` service runs once to set up the database schema.

5. Preparing the Codebase

Let us ensure our Python scripts are clearly named for the pipeline.

1. Verify you have `src/ingestion/get_data.py`.
2. Copy `src/training/lab3_hydra_integration.py` to a new file `src/training/train_model.py` for future use.
3. Create a placeholder for Lab 5 at `src/evaluation/evaluate_model.py`:

```
# src/evaluation/evaluate_model.py
import sys
print("Lab 5: Model Evaluation Logic will be implemented here.")
sys.exit(0)
```

6. DAG Implementation: `dags/weather_pipeline.py`

We will use Airflow's `BashOperator`. Because we integrated Hydra in Lab 3, the `BashOperator` is incredibly powerful: we can dynamically override configurations directly from the DAG definition.

Create your first DAG

Then, create the file `dags/weather_pipeline.py` with the following content:

```
from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
from datetime import datetime, timedelta

# Default arguments applied to all tasks in the DAG
default_args = {
    'owner': 'mlops_engineer',
    'depends_on_past': False,
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

# Define the DAG context
with DAG(
    dag_id='weather_forecast_mlops_pipeline',
    default_args=default_args,
    description='End-to-End MLOps Pipeline for Weather Forecasting',
    schedule='@daily', # Runs once a day automatically
    start_date=datetime(2026, 3, 1),
    catchup=False,
    tags=['weather_capstone', 'training'],
) as dag:
    # Task 1: Data Ingestion
    # We use BashOperator to run our Python script from the /opt/airflow root
    ingest_data = BashOperator(
        task_id='ingest_weather_data',
        bash_command='cd /opt/airflow && python src/ingestion/get_data.py'
    )

    # Task 2: Model Training
    # Notice how we can Leverage Hydra CLI overrides right here in the DAG!
    train_model = BashOperator(
        task_id='train_forecast_model',
        bash_command='cd /opt/airflow && python src/training/train_model.py'
        tracking.experiment_name="Airflow_Automated_Run" tracking.uri="http://mlflow_server:5000"
    )

    # Task 3: Model Evaluation (Lab 5 Placeholder)
    evaluate_model = BashOperator(
        task_id='evaluate_champion_model',
        bash_command='cd /opt/airflow && python src/evaluation/evaluate_model.py'
    )

    # Define the execution Logic (The Graph Edges)
    # Ingestion must succeed before Training, which must succeed before Evaluation
    ingest_data >> train_model >> evaluate_model
```

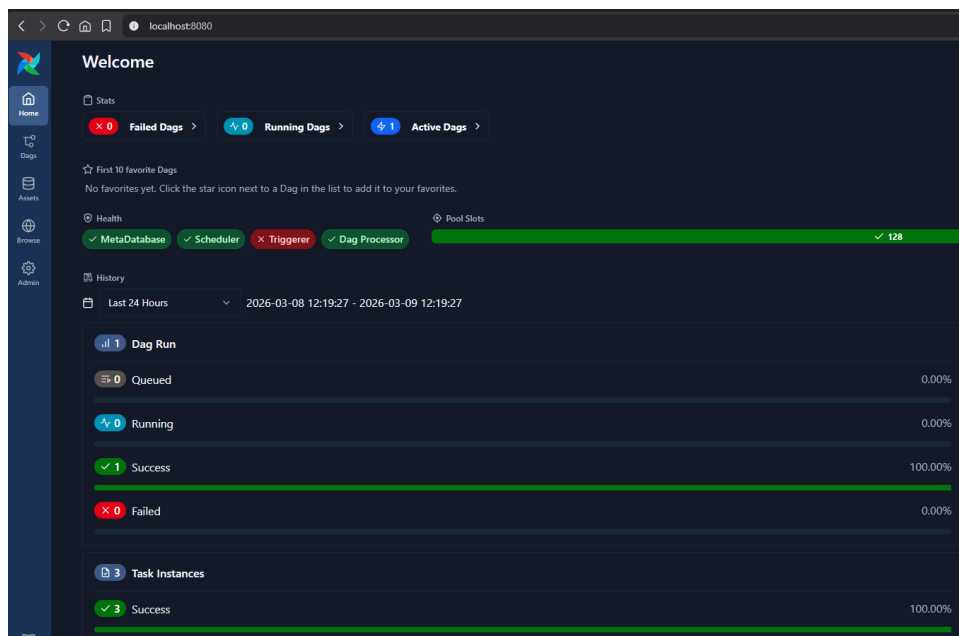
Code Explanation:

- `default_args`: This dictionary defines default parameters for all tasks in the DAG, such as the owner, retry behavior, and email notifications.
- `with DAG(...) as dag`: This context manager defines the DAG itself.
 - `dag_id`: A unique identifier for the DAG.
 - `description`: A brief description of what the DAG does.
 - `schedule`: The schedule on which the DAG should run. Here, `@daily` means it will run once every day. You can customize this to fit your needs (e.g., `@hourly`, `0 0 * * *` for daily at midnight, etc.).
 - `start_date`: The date from which the DAG should start running. Airflow will not run any tasks before this date.

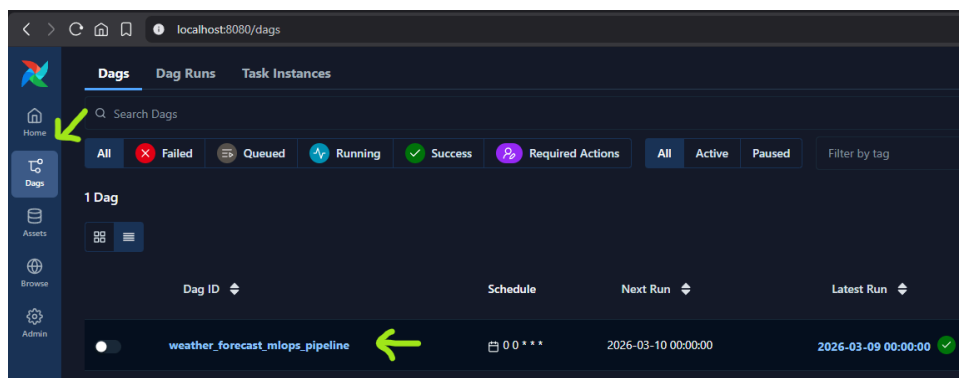
- `catchup` : When set to `False` , Airflow will only run the latest scheduled task and will not backfill any missed runs between the `start_date` and the current date.
 - `tags` : Tags help organize and filter DAGs in the Airflow UI.
- `ingest_data` , `train_model` , and `evaluate_model` : These are the three tasks defined using `BashOperator` . Each task runs a specific Python script from the `/opt/airflow` directory, which is where our project files are mounted in the Airflow container.
 - `task_id` : A unique identifier for the task within the DAG.
 - `bash_command` : The command that will be executed in the shell. We change to the `/opt/airflow` directory to ensure that the Python scripts can access the necessary files and configurations. For the `train_model` task, we also pass Hydra overrides to set the MLflow experiment name and tracking URI, ensuring that the training run is properly logged in our MLflow server.
- `ingest_data >> train_model >> evaluate_model` : This defines the execution order of the tasks. The `>>` operator indicates that `ingest_data` must complete successfully before `train_model` starts, and `train_model` must complete successfully before `evaluate_model` starts. This ensures a linear flow of execution from data ingestion to model training and finally to model evaluation.

7. Running and Monitoring the Pipeline

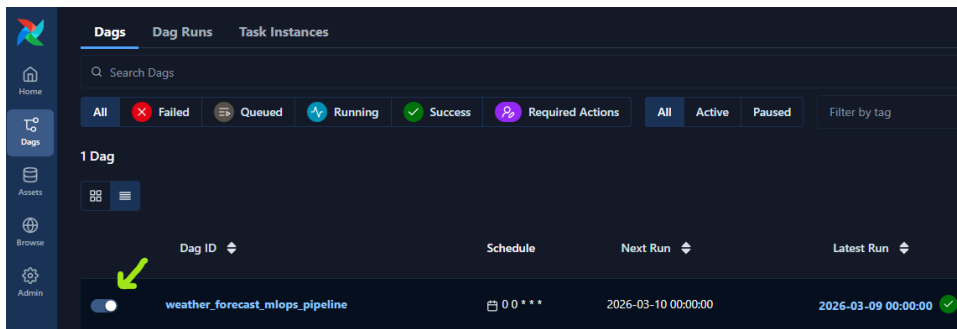
1. Open the Airflow Web UI at `http://localhost:8080` . You should see something like this:



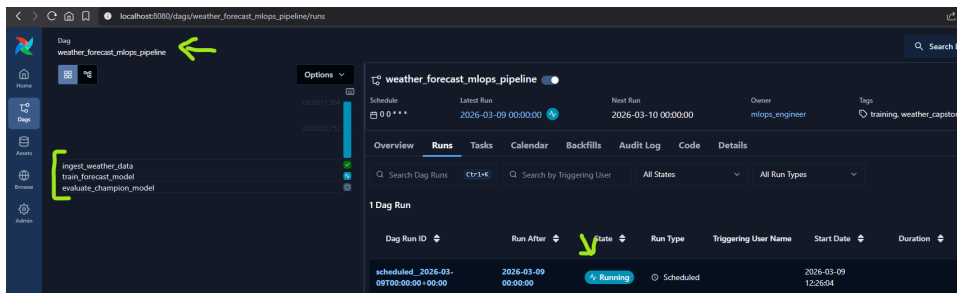
2. Locate `weather_forecast_mlops_pipeline` in the DAGs list.



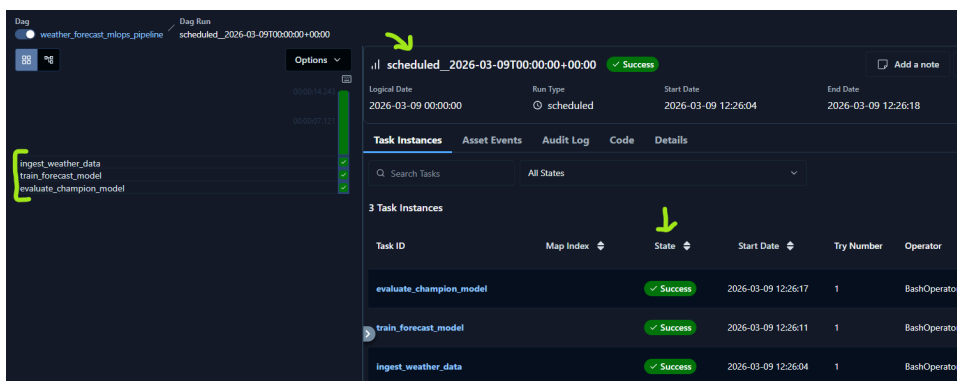
3. Activate the DAG using the toggle switch on the left.



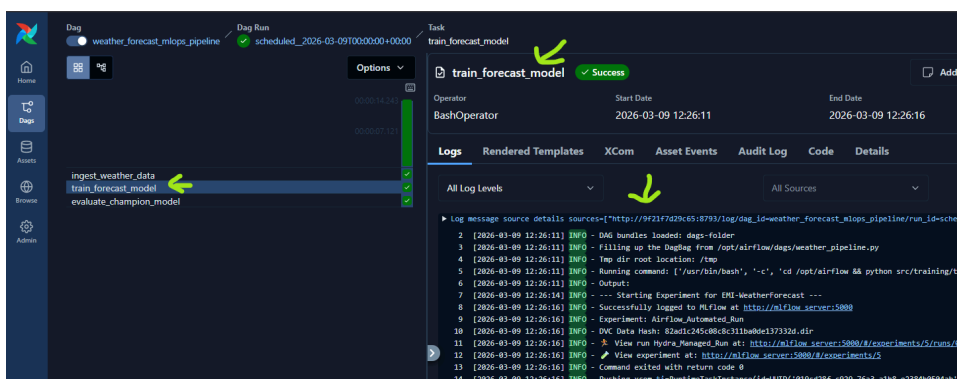
4. Click on the DAG 'weather_forecast_ml_ops_pipeline'. If everything is set up correctly, you should see the DAG run automatically (and still will run daily at the scheduled time).



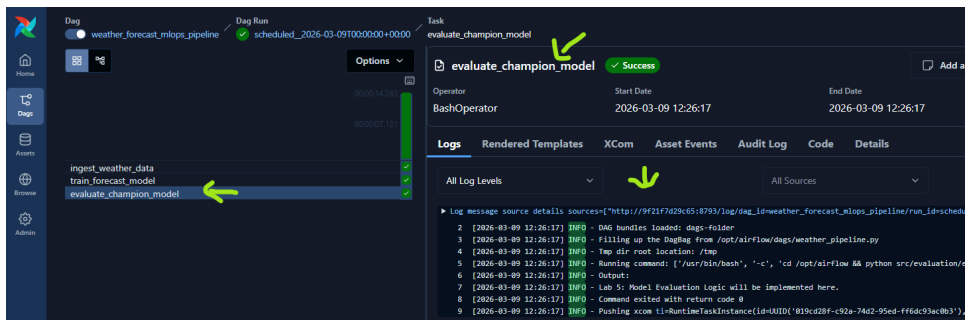
5. Now, if you click on the run 'scheduled__AAAA-MM-DDT:00:00+00:00', you can see the execution details. And if everything ran successfully, you will see green checkmarks for the completed tasks.



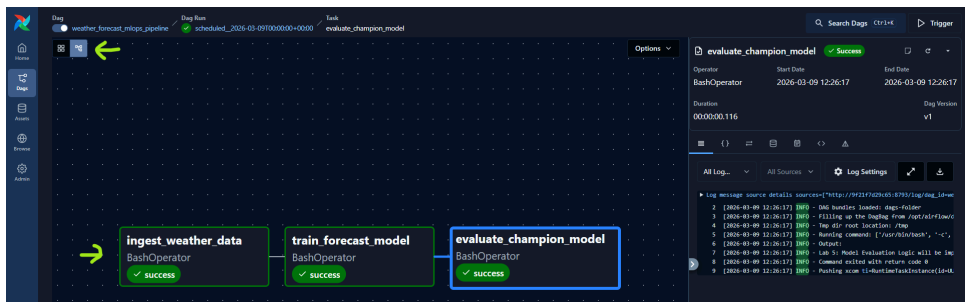
6. You can click on each task to see the logs and details of the execution. For example, clicking on 'train_forecast_model' will show you the logs from the training script, including any output from MLflow logging.



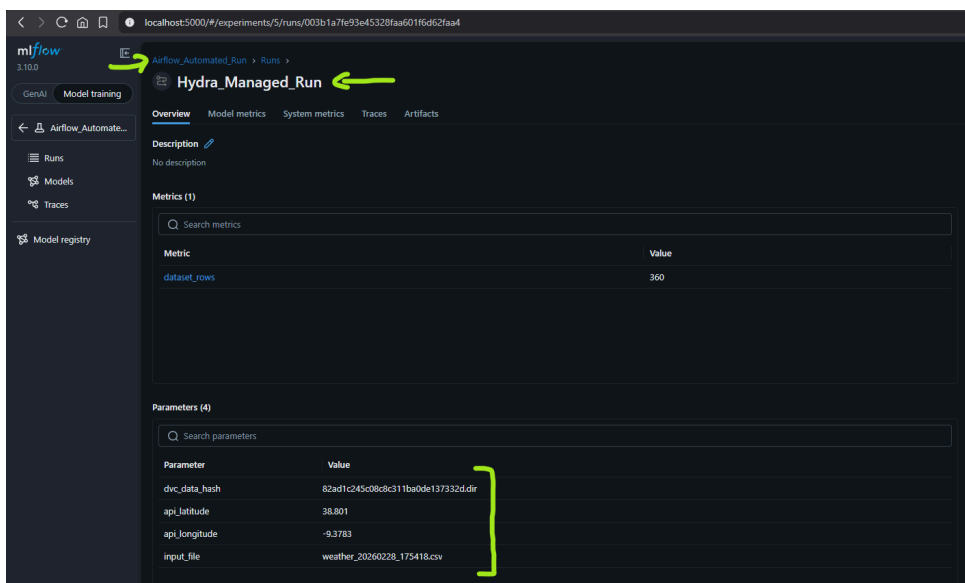
7. You can also click on task 'evaluate_champion_model' to see the placeholder output we set up for Lab 5.



8. To see the Graph View of the DAG, click on the "Graph View" button in the top left corner. This will show you a visual representation of the tasks and their dependencies (showing the linear flow from ingestion to training to evaluation defined by the `>>` operators in our DAG).



9. Finally, you can also check the MLflow UI at <http://localhost:5000> to see the logged experiment from the training task. You should see an experiment named "Airflow_Automated_Run" (which was set via the Hydra override in the DAG) with the corresponding run details.



8. Lab Checklist for Students

- **Infrastructure Check:** Are the `airflow-webserver`, `airflow-scheduler`, and `airflow-dag-processor` containers running without errors? Use `docker ps` to verify (or check your Docker Desktop).
- **DAG Visibility:** Can you see `weather_forecast_mlops_pipeline` in the Airflow UI without import errors?
- **Execution:** Did the DAG execute from start to finish? Check the Airflow logs for the `train_model` task to see the standard output of your script.
- **MLflow Verification:** Did the automated Airflow run successfully push metrics and the DVC data hash to your MLflow tracking server?